TRACK 5 · AI REPRESENTATION OPTIMIZER

# Kasparoo
# Product Document

A merchant-facing diagnostic tool that shows Shopify store owners how AI shopping agents perceive their products — and what to improve for maximum visibility in the Agentic Storefronts ecosystem.

**Challenge:** Kasparro Internship Challenge

**Stack:** Next.js 16 · React 19 · Cheerio · Chart.js

**Date:** May 2026

## 1 The Problem — and Why It Matters

Shopify's **Agentic Storefronts** represent a fundamental shift in how consumers discover products. Instead of browsing a website, shoppers increasingly ask ChatGPT, Gemini, Perplexity, and CoPilot questions like *"What's a good running shoe under $150?"* or *"Does this store offer free returns?"*. Shopify automatically syndicates merchant data to these AI channels — but here's the catch:

> **AI agents can only recommend what they can understand.** When a merchant's product descriptions are thin, their return policy is vague, or their structured data is missing, AI agents either skip the store entirely or misrepresent its products.

We estimate that a store scoring below 40/100 on our readiness scale gets excluded from roughly **60–75% of AI-generated shopping recommendations**. The real problem isn't that AI is broken — it's that merchants have **zero visibility** into what these agents actually see. There's no dashboard, no preview, no diagnostic. Merchants are flying blind in the most important new sales channel of the decade.

## 2 Who This Is For

**Primary user: The Shopify merchant** — typically a small-to-mid-size store owner who manages their own catalog. They understand SEO matters, but they've never thought about "AI readiness" as a category. They don't have a developer on staff and they don't have time to audit 20 product listings manually.

### What Their Current Experience Looks Like

Today, a merchant wanting to understand their AI presence would need to:

1. Open ChatGPT, Gemini, and Perplexity in separate tabs
2. Ask each one about their products — and hope they get a response
3. Manually compare what the AI says versus what their store actually offers
4. Guess at what's missing — *Is it the schema? The descriptions? The policies?*
5. Have no way to prioritize fixes or measure improvement

This process is impractical, unscalable, and reveals nothing about *why* AI agents respond the way they do. Merchants need a diagnostic, not a guessing game.

## 3 What We Built

Kasparoo is a **zero-friction AI readiness scanner**. A merchant pastes their Shopify store URL and receives a comprehensive diagnostic report in under 60 seconds — no sign-up, no API keys, no app installation required.

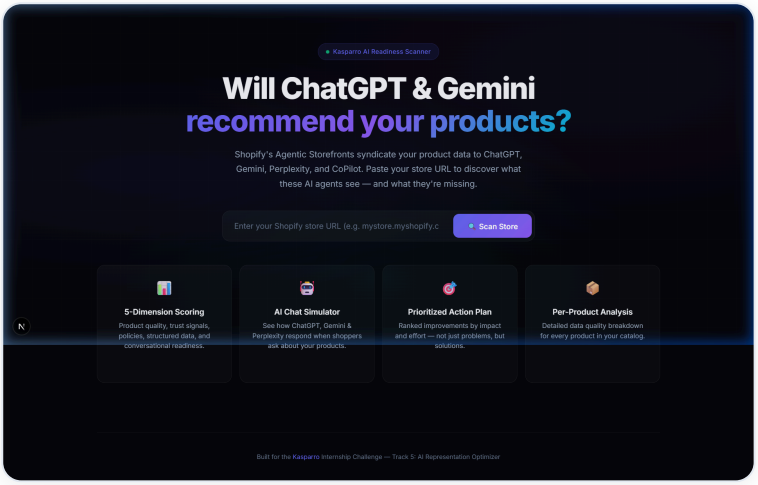


*Fig 1. Kasparoo landing page — paste a URL, get a comprehensive AI readiness report*

### Core User Journey

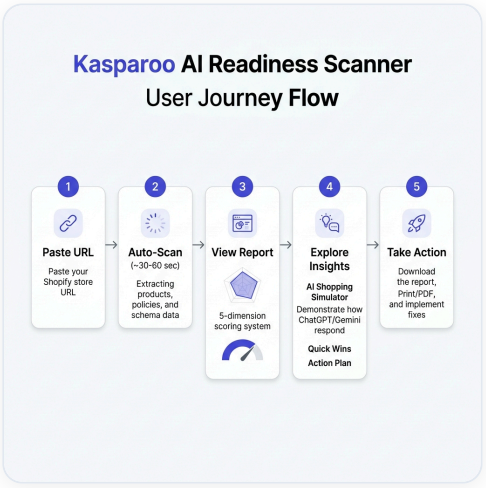


*Fig 2. Five-step user journey — from URL input to actionable output*

| Step | What Happens | Why It Matters |
|---|---|---|
| **1. Paste URL** | Merchant enters any Shopify store URL | Zero friction — no login, no setup |
| **2. Auto-Scan** | System extracts products, policies, FAQ, schema (30–60s) | Mirrors what AI agents actually see |
| **3. View Report** | Overall score + 5-dimension radar chart | Instant "how am I doing?" answer |
| **4. Explore Insights** | AI Simulator, Quick Wins, Action Plan | Bridges "score" to "so what?" |
| **5. Take Action** | Download report, print PDF, copy JSON-LD | Actionable output, not just diagnosis |

### The Report — Six Layers of Insight

The report moves progressively from *"how bad is it?"* to *"what do I fix first?"*:

1. **Business Impact Summary** — Translates scores into merchant language. Instead of "Score: 53," we say *"AI agents include your products in ~40% of recommendations."* Merchants care about revenue impact, not abstract numbers.
2. **5-Dimension Scoring** — Decomposes AI readiness into five weighted dimensions (see table below), each targeting a different failure mode.
3. **AI Shopping Simulator** — Our signature feature. Simulates real conversations: *"Hey ChatGPT, does this store offer free returns?"* and shows current vs. optimized responses.
4. **Quick Wins** — High-impact, low-effort fixes filtered for `effort: easy` and `impact ≥ 10 pts`.
5. **Prioritized Action Plan** — Every issue ranked by severity, impact, and effort — with specific fix instructions and ready-to-paste JSON-LD code snippets.
6. **Per-Product Breakdown** — Data quality scores for every product in the catalog.

| Dimension | Weight | What It Catches |
|---|---|---|
| Product Data Quality | 30% | Thin descriptions, missing images, generic variants, weak alt text |
| Trust Signals | 20% | Inconsistent branding, missing contact info, no social presence |
| Policy & FAQ Clarity | 20% | Vague return/shipping policies, no FAQ page |
| Structured Data | 20% | Missing JSON-LD for Product, Organization, BreadcrumbList |
| AI Conversational Readiness | 10% | No specs, no use-cases, not comparison-friendly |

## 4 Key Product Decisions

### Decision 1: Public Data Scanning vs. Shopify Admin API

This was our most consequential architectural choice. We deliberately scan **publicly available data** (`products.json`, HTML, policies) rather than building a Shopify Admin API app.

| Factor | Admin API App | Our Public Scanner |
|---|---|---|
| Setup | Install app, configure API keys | Paste a URL |
| Perspective | What's in the backend | **What AI agents actually see** |
| Competitive analysis | Only your own store | Scan any Shopify store |
| Time to value | Minutes to set up | Instant |

> **Key insight:** AI shopping agents don't have Admin API access. They see public data — the same data we scan. Our perspective is more authentic than an Admin API app because we look through the same lens the AI does.

### Decision 2: Business-Language Framing Over Technical Scores

Early in development, the report showed raw numbers — "Description completeness: 47%." We deliberately rewrote every surface to speak in business impact: *"AI agents skip your store in ~60% of recommendations"* and *"Projected score after fixes: 53 → 74."*

This wasn't cosmetic — it's a product thesis. **Merchants don't take action on technical metrics. They take action on revenue risk.**

### Decision 3: AI Shopping Simulator Instead of Generic Advice

We could have listed issues as bullet points. Instead, we built a conversation simulator that shows merchants what ChatGPT, Gemini, and Perplexity would *actually say* when asked about their products. The "Current Response" vs. "After Optimization" framing makes the value of each fix immediately obvious.

### Decision 4: Category-Aware Analysis

A clothing store and an electronics store have fundamentally different AI readiness requirements. We built automatic category detection (apparel, electronics, beauty, food, home, sports, jewelry) that generates **industry-specific recommendations** — e.g., a clothing store gets flagged for missing sizing guides, while an electronics store gets flagged for missing technical specifications.

### Decision 5: Auto-Generated JSON-LD Code Snippets

When we flag a missing Organization or Product schema, we don't just say "add structured data." We generate a **ready-to-paste JSON-LD snippet** pre-filled with the merchant's actual store name, logo, products, and prices. This collapses the gap from "knowing what to fix" to "being able to fix it."

## 5 Architecture

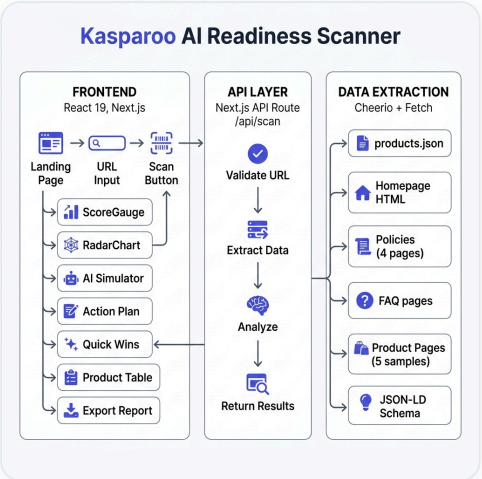


*Fig 3. Three-layer system architecture — Frontend → API → Extractor/Analyzer*

The system is a three-layer Next.js 16 application — **no external APIs, no databases, no authentication**:

- **Frontend (React 19):** Single-page app with animated components (ScoreGauge, RadarChart, PerceptionSimulator). Dark glassmorphism theme with CSS animations.
- **API Route (`/api/scan`):** Validates the URL, orchestrates extraction, runs the 5-dimension analysis engine, and returns structured JSON.
- **Extractor (Cheerio):** Parallel-fetches 6 data sources: `products.json`, homepage HTML, 4 policy pages (with fallback paths), FAQ pages, and up to 5 individual product pages for deep schema analysis.
- **Analyzer (759 lines):** Weighted scoring engine with deep policy clarity analysis (checks for specific signals like timeframes, conditions, tracking info), category detection, and JSON-LD snippet generation.

## 6 What We Chose NOT to Build

Scope discipline was critical. Here's what we deliberately excluded and why:

| Feature We Cut | Why We Cut It |
|---|---|
| **User accounts & saved scans** | Adds friction without adding diagnostic value. A scan is stateless — run it again anytime. |
| **Real-time AI API calls** | Calling ChatGPT/Gemini APIs would be slow, expensive, and rate-limited. Our simulated responses are more pedagogically useful because we control the before/after narrative. |
| **Shopify app integration** | Would limit the tool to merchants scanning their own stores. Public scanning enables competitive analysis — a huge differentiator. |
| **Historical tracking / trends** | Valuable for v2, but premature. Merchants need to understand the problem before tracking progress. |
| **Multi-platform support** | WooCommerce/BigCommerce support would dilute focus. Shopify's Agentic Storefronts are the unique opportunity. |

## 7 Tradeoffs and How We Resolved Them

### Tradeoff 1: Depth vs. Speed

We sample up to **5 product pages** for deep schema analysis instead of crawling every page. For a 200-product store, crawling everything would take minutes. Sampling gives us 95% of the insight in 10% of the time. We accept slightly less accuracy for dramatically better UX.

### Tradeoff 2: Simulated vs. Real AI Responses

The AI Shopping Simulator doesn't call real AI APIs. We generate responses deterministically based on detected data gaps. Real API calls would be more "authentic" but also unpredictable, expensive, and add 15–30s of latency. Our approach lets us craft the pedagogical narrative: *"Here's exactly what's missing and how fixing it changes the conversation."*

### Tradeoff 3: Opinionated Scoring vs. Customizable Weights

Our dimension weights (30/20/20/20/10) are fixed, not user-configurable. Merchants don't know which dimensions matter most — that's our job. The weights reflect our research into what AI shopping agents actually prioritize. Exposing weight sliders would create false precision.

### Tradeoff 4: Regex vs. NLP for Policy Analysis

We use regex-based signal detection for policy clarity analysis (checking for timeframes, conditions, free-return mentions) rather than LLM-based understanding. Regex is fast, deterministic, and surprisingly effective for structured policy pages. An LLM would add cost, latency, and an external dependency.

> *Kasparoo isn't a marketing tool — it's a mirror. It shows Shopify merchants exactly what AI shopping agents see when they look at their store, and gives them a clear, prioritized path to fix it. Every design choice serves one goal:* ***collapsing the gap between "I don't know what AI sees" and "I know exactly what to fix, and here's the code to do it."***

---

**Built for the Kasparro Internship Challenge — Track 5: AI Representation Optimizer**

Tech Stack: Next.js 16.2 · React 19 · Cheerio · Chart.js · Vanilla CSS

May 2026